



The Davis-Putnam-Loveland-Logeman (DPLL) Method

- ▶ Deciding Satisfiability Problems
- ▶ The Rules
- ▶ The DPLL-Algorithm
- ▶ Searching for Models



Deciding Satisfiability Problems

- ▶ Let G be a propositional formula. Is G satisfiable?
- ▶ **Theorem** G is satisfiable iff $\neg G$ is unsatisfiable.
- ▶ Davis-Putnam-Logemann-Loveland Procedure (DPLL)
(Davis, Putnam 1960), (Davis, Logemann, Loveland, 1962).
 - ▷ Given F in CNF.
 - ▷ Is F unsatisfiable?
 - ▷ Several rules and a strategy for applying them.



The Rules

UNSAT If F contains $[\]$, then F is unsatisfiable.

SAT If F is empty, then F is satisfiable.

MULT If a literal occurs more than once in a clause, then all but one can be deleted.

SUBS A clause in F can be deleted, if it is a superset of another clause in F .

UNIT An element $\bar{L}$ of a clause in F can be deleted, if F contains $[L]$.

TAUT A clause in can be deleted, if it contains a literal and its complement.

PURE A clause can be deleted, if it contains L and $\bar{L}$ does not occur in F .

SPLIT If F is semantically equivalent to a formula of the form

$$\langle (C_1 \vee L), \dots (C_k \vee L), (C_{k+1} \vee \bar{L}), \dots (C_m \vee \bar{L}), C_{m+1}, \dots, C_n \rangle$$

where neither L nor $\bar{L}$ occur in $C_i, 1 \leq i \leq n$,

then replace F by the CNF of

$$\langle C_1, \dots, C_k, C_{m+1}, \dots, C_n \rangle \vee \langle C_{k+1}, \dots, C_m, C_{m+1}, \dots, C_n \rangle.$$



Examples

UNSAT $\langle [], C_1, \dots, C_n \rangle \equiv []$.

MULT $[L, L, L_1, \dots, L_m] \equiv [L, L_1, \dots, L_m]$.

SUBS $\langle [L_1, \dots, L_m], [L_1, \dots, L_m, \dots L_k], C_1, \dots, C_n \rangle$
 $\equiv \langle [L_1, \dots, L_m], C_1, \dots, C_n \rangle$.

UNIT $\langle [L_1, \dots, L_m, L], [\bar{L}] \rangle \equiv \langle [L_1, \dots, L_m], [\bar{L}] \rangle$.

TAUT $\langle [L_1, \dots, L_m, L, \bar{L}], C_1, \dots, C_m \rangle \equiv \langle C_1, \dots, C_m \rangle$.

PURE $\langle [p, \neg q], [\neg r] \rangle \not\equiv \langle [p, \neg q] \rangle$,

but $\langle C_1, \dots, C_m, [L, L_1, \dots, L_n] \rangle$ is unsatisfiable

iff $\langle C_1, \dots, C_m \rangle$ is unsatisfiable,

where $\bar{L}$ does neither occur in $C_i, 1 \leq i \leq m$ nor in $L_j, 1 \leq j \leq n$.

SPLIT $\langle [p, r], [\neg r], [q] \rangle \not\equiv \langle [p, q], [q, q] \rangle$,

but the rule preserves unsatisfiability.



Some Properties of the Rules

- ▶ Let F be a formula in CNF and F' be obtained from F by applying a rule.
- ▶ MULT, SUBS, UNIT and TAUT are equivalence preserving, i.e., $F' \equiv F$.
- ▶ They are polynomial simplification rules.
- ▶ PURE and SPLIT are unsatisfiability preserving, i.e., F is unsatisfiable **iff** F' is unsatisfiable.



The DPLL-Algorithm

- 1 Input F .
- 2 Transform F into CNF.
- 3 Apply the rules MULT, SUBS, UNIT, TAUT, PURE and SPLIT until SAT or UNSAT become applicable.
- 4 If SAT is applicable then terminate with “ F is *satisfiable*.”
- 5 If UNSAT is applicable then terminate with “ F is *unsatisfiable*.”

Remarks

- ▷ The algorithm always terminates.
- ▷ Whenever the application of a rule in step 3 yields a formula H , then H is unsatisfiable **iff** F is unsatisfiable.
- ▷ The algorithm is sound and complete.



Strategy

- ▶ A **strategy** determines the selection of the applied rule in step 3.
- ▶ A “*good*” strategy:
 - ▷ Apply MULT, SUBS, UNIT, TAUT, PURE eagerly.
 - ▷ Make a “wise” decision concerning the literal to be eliminated in an application of SPLIT.



An Example

$\langle [p_1, p_2], [p_4, \neg p_2, \neg p_3], [p_1, p_3], [\neg p_4] \rangle$

Initialization

$\langle [p_1, p_2], [\neg p_2, \neg p_3], [p_1, p_3], [\neg p_4] \rangle$

UNIT wrt $[\neg p_4]$

$\langle [p_1, p_2], [\neg p_2, \neg p_3], [p_1, p_3] \rangle$

PURE wrt $\neg p_4$

$\langle [p_2, \neg p_2, \neg p_3], [p_3, \neg p_2, \neg p_3] \rangle$

SPLIT wrt p_1 .

$\langle [p_3, \neg p_2, \neg p_3] \rangle$

TAUT wrt $[p_2, \neg p_2, \neg p_3]$

$\langle \rangle$

TAUT wrt $[p_3, \neg p_2, \neg p_3]$

“satisfiable”

SAT



Searching for Models

- ▶ Let G be a propositional formula. Is G satisfiable? If it is, then find a model.
- ▶ MULT, SUBS, UNIT and TAUT can be applied to G yielding semantically equivalent formulas.
- ▶ If G contains a unit clause $[L]$, then for each model I for G we find $L^I = \top$.
- ▶ If PURE is applicable to G wrt L yielding G' , then G is satisfiable **iff** $G' \wedge L$ is satisfiable.
- ▶ If F is semantically equivalent to a formula of the form

$$\langle (C_1 \vee L), \dots, (C_k \vee L), (C_{k+1} \vee \bar{L}), \dots, (C_m \vee \bar{L}), C_{m+1}, \dots, C_n \rangle$$

where neither L nor $\bar{L}$ occur in C_i , $1 \leq i \leq n$, then F is satisfiable **iff** either

$$\langle C_1, \dots, C_k, C_{m+1}, \dots, C_n, [\bar{L}] \rangle$$

or

$$\langle C_{k+1}, \dots, C_m, C_{m+1}, \dots, C_n, [L] \rangle$$

is satisfiable.



The Search Rules

- ▶ Let J be a set of literals.
- ▶ **sRULE** If F' is obtained from F by applying $\text{RULE} \in \{\text{MULT}, \text{SUBS}, \text{UNIT}, \text{TAUT}\}$, then (F', J) is obtained from (F, J) by applying sRULE.
- ▶ **sPURE** If F' is obtained from F by applying PURE wrt L , then $(F', J \cup \{L\})$ is obtained from (F, J) by applying sPURE.
- ▶ **sSPLIT** If $\langle C_1, \dots, C_k, C_{m+1}, \dots, C_n \rangle \vee \langle C_{k+1}, \dots, C_m, C_{m+1}, \dots, C_n \rangle$ is obtained from F by applying SPLIT wrt L , then either $(\langle C_1, \dots, C_k, C_{m+1}, \dots, C_n \rangle, J \cup \{\bar{L}\})$ or $(\langle C_{k+1}, \dots, C_m, C_{m+1}, \dots, C_n \rangle, J \cup \{L\})$ is obtained from (F, J) applying sSPLIT.

Remarks

- ▶ sSPLIT is don't-know non-deterministic.



A Simple Search Algorithm

- 1 Input G .
- 2 Let F be the CNF of G .
- 3 Set $J := \emptyset$.
- 4 Apply the rules sMULT, sSUBS, sUNIT, sTAUT, sPURE and sSPLIT updating (F, J) until SAT or UNSAT become applicable to F .
- 5 If SAT is applicable,
then terminate with “ G is satisfiable” and generate a model from J .
- 6 If UNSAT is applicable to all leaves on the branches wrt sSPLIT,
then terminate with “ G is unsatisfiable”.

Remarks

- ▷ The algorithm terminates, is sound and complete.
- ▷ J never contains a literal together with its complement.



DPVis: Visualizing the DPLL Procedure

- ▶ An **interaction graph** is an undirected graph that contains one node for each propositional variable and an arc connecting any two nodes whose associated variables appear in the same clause.
- ▶ Clauses with two literals are specifically highlighted by marking the arc between the nodes representing the propositional variable occurring in the clause.
 - ▷ Two positive literals: red, double-ended arrow.
 - ▷ Two negative literals: green, double-ended arrow.
 - ▷ A positive and a negative literal: blue arrow from the node representing the propositional variable occurring in the negative literal to the node representing the positive literal.